INFERENCE

Inference is the process of using a trained model to predict biological labels associated with your biological question for new, previously unseen images.

Inference analysis requires two things:

- A trained model – built and saved in advance from your training dataset using the "Train model" option.
- Features from the new images – the unknown images must first be passed through feature extraction by running "Generate features" in unlabelled mode.

The biological question you are posing already matters at the level of experimental design and at the analysis stage (especially for feature-wise interpretation), so consider the following points before proceeding with inference:

File naming. To make this easier, ensure every image follows the exact naming convention, including information on the date (independent experiment) and technical repetition, such as well and measurement position, so that you can trace any problems. The image name should also contain the information you need to track the biological question, e.g. treatment type, identity of the new strain, etc. We suggest that you follow the naming convention used for the training data and add _treat_ at the end of the name that contains information on the treatment, so that parsing the data from the name is easy and enables automation of analysis.

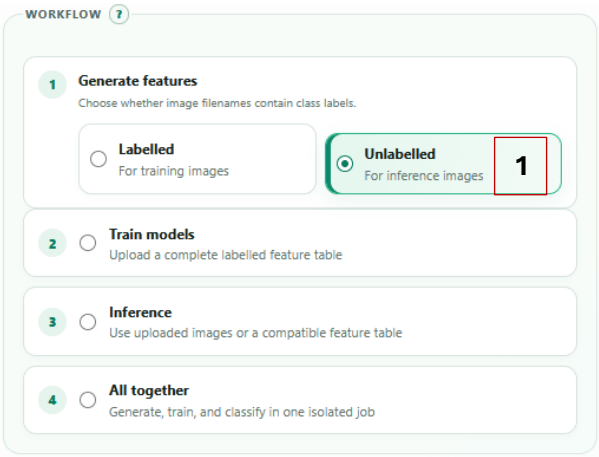
Recommendation for "treatment"-related biological questions. If you are predicting how a treatment alters structure, always include untreated controls in both the experiment and the analysis. Always perform several technical replicates and independent experiments on different dates for each treatment and control. This makes interpretation more reliable.

If you are using untreated strains as training data to ask whether their structure has changed, then during the analysis split the experimental controls into two parts. One part should include control images for the training set, so that the model becomes familiar with the diversity introduced by the additional natural variability of the controls, and the second part should be included in the inference sets as sanity checks that "validate" the model's performance and support interpretation. You should perform this split manually before running inference. The controls in the inference sets are your positive control: if the model cannot classify held-out control images correctly, this may indicate a failure and requires troubleshooting.

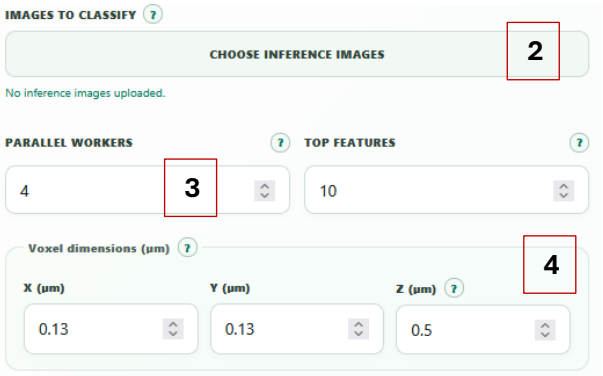
Warning! The set of images you choose to run in the inference module as a batch is critical. The outputs, particularly the feature-based output, depend on the entire dataset provided for inference, not, for example, on each treatment individually. This means that if you have carried out 10 treatments and want to compare these as a dataset, you should run inference on the full dataset so that the effects of all treatments are analysed together. This is recommended to gain an overview of the dataset as a whole, including the controls as a sanity check. Afterwards, it is advisable to run inference solely on individual treatments to obtain information on each of them.

Inference consists of two steps: first, calculate features from the new images to obtain a list of features that the model can then use to make an inference on them, which is done in the second step.

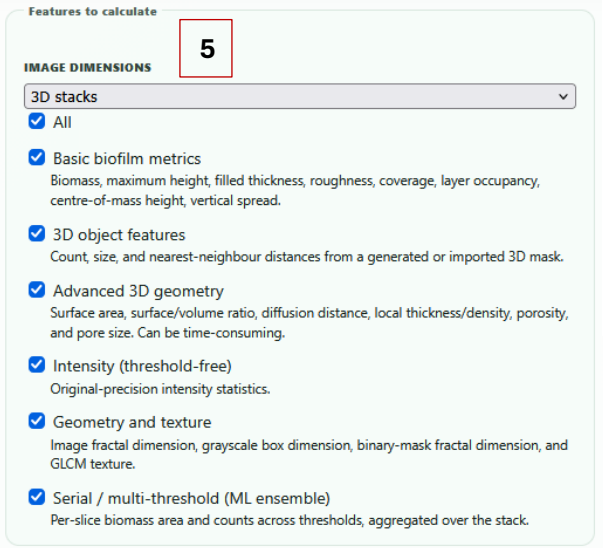
Generate features for the unknown images



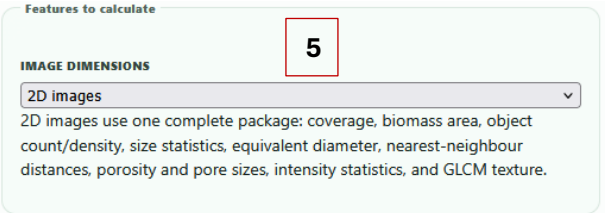
1. Choose “Generate features” in mode “Unlabelled”.

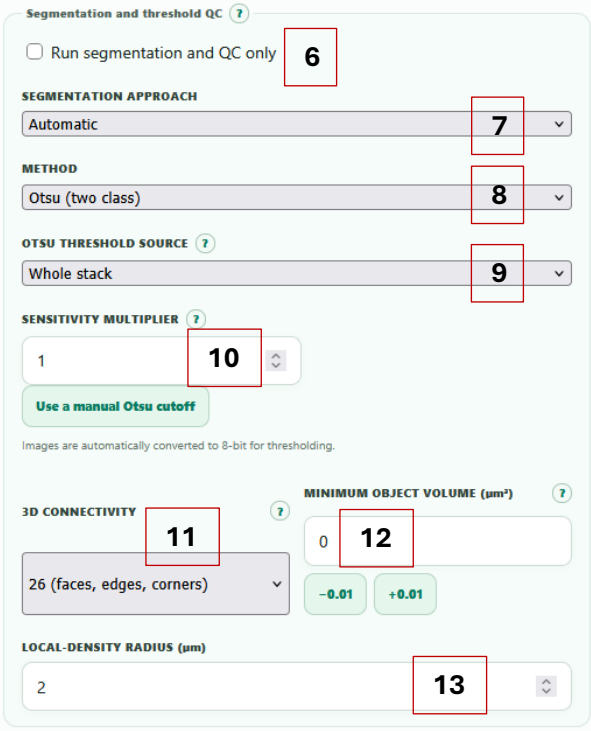


2. Point the software to the folder containing your images in tif format, using “Choose inference images”.
3. Set the running options. “Parallel workers” – how much of your CPU to dedicate to the analysis: use full capacity when you are not otherwise using the computer, and less if you are working on it at the same time. Suggested workers will provide optimal settings for running.
4. Adjust the voxel dimensions (the pixel size in micrometres) to match your images.



5. Choose the type of image you will supply to the software (2D or 3D).
For 2D images, a predefined subset of features is calculated automatically. For 3D images, the available feature groups are displayed and can be selected individually (ticked) according to the descriptors you require.





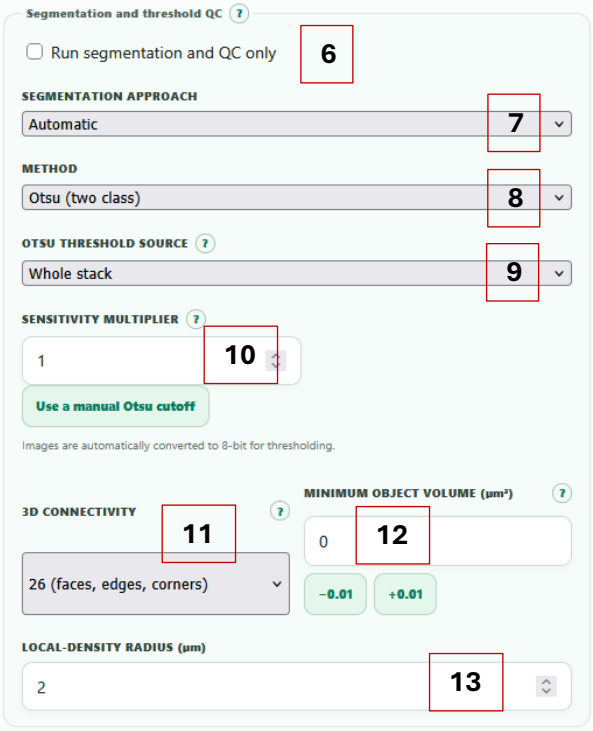
6. (Optional) Run segmentation and QC only – tick this to produce only the segmentation masks, without calculating features. This is useful when testing different algorithms and optimising biomass detection, and is ideally done on a representative subset of images.

7. Segmentation approach – choose how the biofilm mask is obtained:

- Automatic – the threshold is computed automatically, using the method selected in step 8.
- Manual – the user supplies a fixed threshold, which can be optimised beforehand in ImageJ.
- Import – load pre-made masks (e.g. from machine learning or single-cell segmentation) as tif files, bypassing the built-in thresholding entirely.

8. Method – the thresholding algorithm used in Automatic mode:

- Otsu (two class) — standard Otsu thresholding, which splits the intensity histogram into two classes (biofilm vs background) by maximizing the between-class variance. A good general default when the biofilm is clearly brighter than the background.
- Otsu (three class) — multi-level Otsu, which splits the histogram into three classes (background / dim / bright). Useful when the image contains two intensity populations (e.g. bright cells and dimmer material); the intermediate “dim” class can be assigned to either biomass or background.
- Robust background (two class) — estimates the background statistically as its mean + 2σ (after trimming the intensity tails) rather than from the full histogram. More robust when the background is noisy or has a baseline offset.



9. Otsu threshold source – the region used to compute the automatic threshold:
   - Whole stack – the threshold is derived from all layers and applied to the whole volume.
   - Substratum – the threshold is derived from the first (substratum) layer. This follows the COMSTAT convention.

10. Sensitivity multiplier – a factor applied to the automatically computed threshold to fine-tune how stringent the segmentation is (default 1).

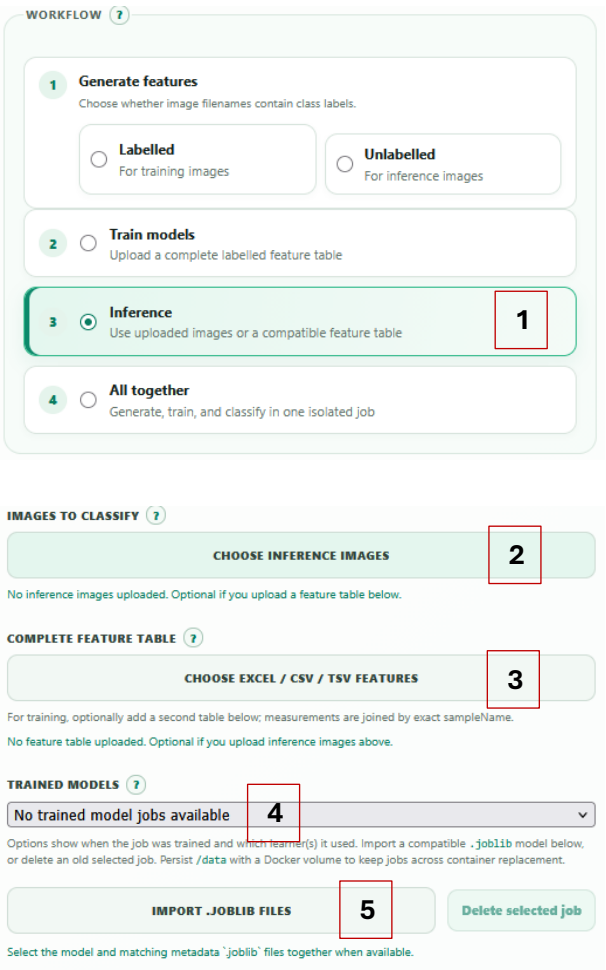
11. Minimum object volume (µm³) — removes connected objects smaller than this volume (default 0, meaning keep all). Use it to exclude noise; a biologically meaningful value is roughly half a single-cell volume, estimated from cell dimensions measured in ImageJ. Adjust with the −0.01 / +0.01 buttons.

12. 3D connectivity – defines when neighboring voxels belong to the same object: 6 (shared faces only), 18 (faces and edges), or 26 (faces, edges, and corners; default). Higher connectivity merges voxels more readily, giving fewer, larger objects.

13. Local-density radius (µm) — the radius of the sphere used to compute local biomass density around each voxel (default 2 µm). Values are comparable across images only when the same radius is used.

Click “Run pipeline”.

Run inference on unknown images



1. Choose “Inference”.

2. Point the software either to the folder containing the images to be classified, using “Choose inference images”, or directly to a feature table (the features generated in unlabelled mode), using “Choose excel/tsv/csv features”.

3. (Optional) Upload additional features as a .tsv file. The sample names must match those in the feature table selected in step 1, so that the two files can be merged. If the model was trained with additional (external) features, those features must also be supplied here for inference.

4. Select the trained model to apply, using the “Trained models” drop-down menu. All trained models are stored in the Docker folder, so the appropriate one is chosen from this drop-down. Models are named by the date they were trained, followed by the number of models in the job (one, if a single learner was selected, or six, if all learners were selected), and a Docker-generated code used for tracking in case of problems. To remove an old or test model, select it from the drop-down and click “Delete selected job”. However, to avoid taking up too much space, the jobs of the MicroICS may be deleted when a new job is set. Thus, use the models in point 5.

5. To add models from elsewhere, click “Import _joblib files”, browse to the folder containing the models, and select those you wish to import (one or several). Once imported, they are available for inference.

Set “Parallel workers” as in training; “Suggested workers” provides a sensible default for your machine.

Click “Run pipeline”.

Understanding the inference output

The most important results are the prediction (by majority vote), the probability behind it, and the feature-level explanation that supports interpretation. For each trained model, separate output files are provided, and below we give an example for the random forest.

List of outputs:

- **inference_summary** – an overview of the run
  The columns of the table:
  Model – the model used; rf denotes random forest.
  Num_predictions – the total number of images classified.
  Unique_predictions – the number of distinct classes predicted across the set.

- **rf_predictions** – the predicted label for each image; the primary output for seeing which class each new image was assigned to.
  The columns of the table:
  Sample_name – the image identifier.
  Prediction – the class (strain) the model assigned to the individual image

- **rf_probabilities** – the confidence behind each prediction.
  For each class in the training set, a probability indicates how likely the unknown sample is to belong to that class. Values near 1 indicate a good match; low values indicate that the image resembles more than one class or a structure not in the training set. The probabilities sum to 1 and reflect the proportion of random forest trees voting for each class. When several classes have similar probabilities, the prediction is ambiguous, and the feature-level visualisations should be examined to understand the model's decision.

- **rf_features** – the feature values calculated for the new images, used to compare their profiles with the training set (this is the same as unknown.tsv run in "generate features" mode "unlabelled").

Folder "Explanations":

- **rf_feature_importance** – feature importance across the whole inference set (the inference-set counterpart of rankings_label).

- **rf_shap_values** – For each image, every feature with its SHAP value, which is the contribution of each feature to that image's predicted class.
  For every image, plots derived from SHAP (SHapley Additive exPlanations) values are provided. SHAP quantifies how much each feature contributed to a prediction: a positive SHAP value means the feature pushed the model towards a given class; a negative value means it pushed away.

- **rf_summary_plot:** This plot shows how multiple structural features influence predictions across all submitted images. Each dot represents one image for a given feature, positioned along the x-axis by the strength and direction of that feature's influence on the predicted class. Because each image's class is decided individually by majority vote, all images appear together on one plot, each dot reflecting its own prediction.
  Clusters of similarly coloured dots on one side indicate that the model relies on the magnitude of those features.

- **rf_feature_importance_class:** A bar chart for each class showing which features most strongly influenced the prediction for that class. Only the top features are shown.

- **rf_shap_class:** Per-image SHAP values are provided in the table, which forms the basis for the summary plots described below.

- **rf_shap_plot:** Similar to the beeswarm plot above, but generated separately for each class.

- For each image, a separate plot is produced for every class in the training data, so all categories can be compared. These are most useful in two situations:
  When two classes have similar probabilities – the plots reveal which features may have caused the confusion.
  When the predicted class differs from the true class (e.g. L628 predicted as L1764) — the plots show which features drove the switch, compared with the original class.

Interpreting inference results: a recommended workflow

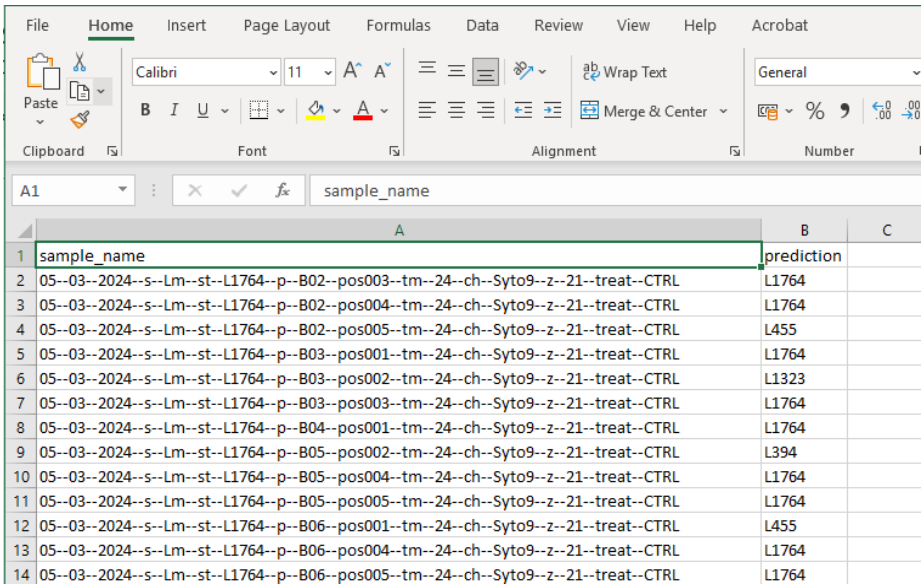
The inference results are always read in the following order: 1. predictions, 2. probabilities, 3. SHAP values/feature values and importance.

The interpretation of the inference results really depends on the biological question. Here are two use cases described to help with setting up the interpretation:

- Question: does the treatment of biofilm move the structure of the biofilm? In this case, we have the "true" identity of the strain that was treated and we compare the untreated strain with the treatment. The control of the experiment is the reference, and you compare it with the results of the treatment. You primarily look for feature shifts in treated vs control and how the prediction and probabilities changed. At feature level, you search for features that changed, in which direction, and by how much between the control and treatment. You interpret the structural shift towards a class using the probabilities, and the feature values moved towards or away from the classes.
- Question: You find a strain with interesting properties and want to determine whether its biofilm structure resembles one of the established classes (grouped as persistent vs non-persistent, clinically relevant or not, etc.). Here you do not know the true label, in contrast to the above example. You primarily rely on the prediction and probability. You examine the SHAP plots to see which features make it resemble each class and compare feature values between the unknown strain and strains of that class. Be aware that the model is "forced" to pick one strain, thus the probability and visual check of congruence between the prediction and the picked class is necessary to confirm the results.

Evaluation of prediction outcomes:

- **Aggregate the results**. After inference, gather all replicates of the same treatment condition and build a matrix counting inferred classes per treatment (Table 1), drawn from rf_predictions (Figure 1). The predominant class per condition is the one with the highest count in its row – aggregating across replicates in this way increases confidence beyond any single image.
  Check whether the controls in the experiment were correctly predicted as a sanity check. The predictions and probabilities for the controls should be correct relative to the true label.



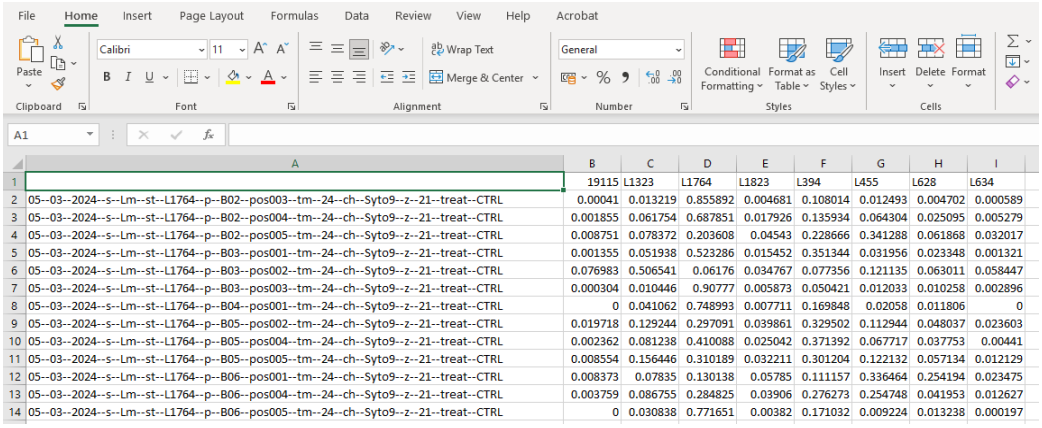
	A	B	C
1	sample_name	prediction	
2	05--03--2024--s--Lm--st--L1764--p--B02--pos003--tm--24--ch--Syto9--z--21--treat--CTRL	L1764	
3	05--03--2024--s--Lm--st--L1764--p--B02--pos004--tm--24--ch--Syto9--z--21--treat--CTRL	L1764	
4	05--03--2024--s--Lm--st--L1764--p--B02--pos005--tm--24--ch--Syto9--z--21--treat--CTRL	L455	
5	05--03--2024--s--Lm--st--L1764--p--B03--pos001--tm--24--ch--Syto9--z--21--treat--CTRL	L1764	
6	05--03--2024--s--Lm--st--L1764--p--B03--pos002--tm--24--ch--Syto9--z--21--treat--CTRL	L1323	
7	05--03--2024--s--Lm--st--L1764--p--B03--pos003--tm--24--ch--Syto9--z--21--treat--CTRL	L1764	
8	05--03--2024--s--Lm--st--L1764--p--B04--pos001--tm--24--ch--Syto9--z--21--treat--CTRL	L1764	
9	05--03--2024--s--Lm--st--L1764--p--B05--pos002--tm--24--ch--Syto9--z--21--treat--CTRL	L394	
10	05--03--2024--s--Lm--st--L1764--p--B05--pos004--tm--24--ch--Syto9--z--21--treat--CTRL	L1764	
11	05--03--2024--s--Lm--st--L1764--p--B05--pos005--tm--24--ch--Syto9--z--21--treat--CTRL	L1764	
12	05--03--2024--s--Lm--st--L1764--p--B06--pos001--tm--24--ch--Syto9--z--21--treat--CTRL	L455	
13	05--03--2024--s--Lm--st--L1764--p--B06--pos004--tm--24--ch--Syto9--z--21--treat--CTRL	L1764	
14	05--03--2024--s--Lm--st--L1764--p--B06--pos005--tm--24--ch--Syto9--z--21--treat--CTRL	L1764	

Figure 1. Screenshot of the rf_predictions output file in TSV format, which can be opened using Excel.

Table 1. Summary of predicted class counts for each treatment condition.
This table shows the number of images classified as belonging to each class (strain) for repeated experiments under the same condition (combination of strain and treatment). Each row represents one treatment condition with multiple replicates. The numbers indicate how many images the model assigned to each class.

		voted class							
strain	treatment	L1764	L455	L1323	L394	L628	ATCC 19115	L1823	L634
L1764	control	19	2	1	3	0	0	0	0
L1764	melon extract coat	46	0	0	4	0	0	0	0
L1764	milk coat	46	0	0	2	0	0	2	0
L1764	tatar steak extract coat	50	0	0	0	0	0	0	0
L628	control	0	0	0	0	24	0	1	0
L628	melon extract coat	21	14	0	7	5	0	2	0
L628	milk coat	42	0	0	4	0	0	4	0
L628	tatar steak extract coat	47	0	0	1	0	0	2	0

- **Read the probabilities**. Each prediction carries a confidence score (rf_probabilities – Figure 2):



	A	B	C	D	E	F	G	H	I
1		19115	L1323	L1764	L1823	L394	L455	L628	L634
2	05--03--2024--s--Lm--st--L1764--p--B02--pos003--tm--24--ch--Syto9--z--21--treat--CTRL	0.00041	0.013219	0.855892	0.004681	0.108014	0.012493	0.004702	0.000589
3	05--03--2024--s--Lm--st--L1764--p--B02--pos004--tm--24--ch--Syto9--z--21--treat--CTRL	0.001855	0.061754	0.687851	0.017926	0.135934	0.064304	0.025095	0.005279
4	05--03--2024--s--Lm--st--L1764--p--B02--pos005--tm--24--ch--Syto9--z--21--treat--CTRL	0.008751	0.078372	0.203608	0.04543	0.228666	0.341288	0.061868	0.032017
5	05--03--2024--s--Lm--st--L1764--p--B03--pos001--tm--24--ch--Syto9--z--21--treat--CTRL	0.001355	0.051938	0.523286	0.015452	0.351344	0.031956	0.023348	0.001321
6	05--03--2024--s--Lm--st--L1764--p--B03--pos002--tm--24--ch--Syto9--z--21--treat--CTRL	0.076983	0.506541	0.06176	0.034767	0.077356	0.121135	0.063011	0.058447
7	05--03--2024--s--Lm--st--L1764--p--B03--pos003--tm--24--ch--Syto9--z--21--treat--CTRL	0.000304	0.010446	0.90777	0.005873	0.050421	0.012033	0.010258	0.002896
8	05--03--2024--s--Lm--st--L1764--p--B04--pos001--tm--24--ch--Syto9--z--21--treat--CTRL	0	0.041062	0.748993	0.007711	0.169848	0.02058	0.011806	0
9	05--03--2024--s--Lm--st--L1764--p--B05--pos002--tm--24--ch--Syto9--z--21--treat--CTRL	0.019718	0.129244	0.297091	0.039861	0.329502	0.112944	0.048037	0.023603
10	05--03--2024--s--Lm--st--L1764--p--B05--pos004--tm--24--ch--Syto9--z--21--treat--CTRL	0.002362	0.081238	0.410088	0.025042	0.371392	0.067717	0.037753	0.00441
11	05--03--2024--s--Lm--st--L1764--p--B05--pos005--tm--24--ch--Syto9--z--21--treat--CTRL	0.008554	0.156446	0.310189	0.032211	0.301204	0.122132	0.057134	0.012129
12	05--03--2024--s--Lm--st--L1764--p--B06--pos001--tm--24--ch--Syto9--z--21--treat--CTRL	0.008373	0.07835	0.130138	0.05785	0.111157	0.336464	0.254194	0.023475
13	05--03--2024--s--Lm--st--L1764--p--B06--pos004--tm--24--ch--Syto9--z--21--treat--CTRL	0.003759	0.086755	0.284825	0.03906	0.276273	0.254748	0.041953	0.012627
14	05--03--2024--s--Lm--st--L1764--p--B06--pos005--tm--24--ch--Syto9--z--21--treat--CTRL	0	0.030838	0.771651	0.00382	0.171032	0.009224	0.013238	0.000197

Figure 2. Screenshot of the rf_probabilities output file in TSV format, viewable in Excel. Column headers include the inferred image names and the names of each class from the training set.

- High probabilities indicate strong confidence. Probabilities around or below 0.6 warrant careful examination as they may signal ambiguous or difficult cases. A low probability often means the model has assigned the

nearest available training class, but with low confidence because the sample does not closely resemble any of them.

- Some strains are inherently harder to distinguish than others. Identify these by examining the confusion matrix from the "Train a model" step: a low-confidence or unexpected prediction is less surprising for strains the model already struggles to separate.
- Compile the probabilities for all replicates of each treatment and summarise them (mean, median, standard deviation, range, percentiles). This shows the central tendency and spread of the model's confidence per treatment and flags outliers; the average probability then guides interpretation.

- **Reading a predicted class that differs from the known one:** In an inference experiment, you sometimes know the true identity of your images – for example, that a given set shows strain L628 under a food-extract treatment. When the model predicts a different class – say, L628 classified as L1764 – this suggests the treated biofilm structure has shifted enough to resemble another strain. If the predicted probability is high, the resemblance is strong based on the calculated features. If the probability is low, it means this is simply the closest example to which the structure has moved in the model's feature space, but it may not actually be very similar. It is just the way the model reports that the structure has changed and which structure it most closely resembles, while effectively indicating that it has encountered something it has not seen before (was not trained on). In this context, analysing the features and contributions described below will help.

Focusing on the features that explain which features changed in the inference dataset

Feature interpretation is always performed with regard to the dataset used during the inference run, as a batch. For example, if you submit a dataset with controls and different treatments at once, the outputs are tailored to the "mixed" dataset. However, if you submit only one type of treatment, the output is tailored to that treatment.

We advise running both: first altogether to obtain an overview of the dataset, and then treatment by treatment to obtain the necessary details.

Key output files and analysis to do:

1. **rf_features:** The feature values calculated for each image used for prediction. This is the same as the input unknow.tsv

Folder "explanations":

2. **rf_feature_importance – table:** Feature importance for the overall dataset (all classes combined). Each feature receives a value and is ranked based on this value to show how important an individual feature was for the entire dataset. This is similar to what ranking_label reports after training. Choose the top 20 features (or more). These features should be cross-checked with rankings_date from the training to exclude or mark features that are important to predict the date. Are these top 20 features similar to those from the training?

3. Filter the feature values of the selected top 20 features to answer the following questions: Is there a difference in feature values between controls and treatments? Is there a difference in feature values when comparing true and predicted classes? How do feature values affect the predicted class?

Based on the gathered information, examine the representative images of controls and treatments. Do they match? If you observe any non-congruent results, refer to the SHAP values to determine which features influenced the decision for each image where non-congruent results were observed. If you find that important features visible in the images are not captured by the calculated features, devise new features to include in the calculation to address this.

4. **rf_shap_class (explanations)**: Per-image SHAP values are provided in the table, which forms the basis for the summary plots described below. Collecting SHAP values for each feature helps you understand problems in inference in cases of non-congruent results.

5. **Summary_plot_class (explanations)**: The beeswarm (per class) shows, for each feature, the spread of SHAP values across images and how the feature's value (colour) relates to its SHAP direction.
   Option 1: high values (red) consistently push in one direction, low values (blue) in the other – colour and SHAP position are ordered. This means the feature has a consistent, monotonic effect: more of it reliably pushes towards the class. Option 2: red and blue dots mixed on both sides – the feature's value does not consistently relate to its effect. High values sometimes push towards, sometimes away. This is a less reliable feature – its effect depends on context or interactions.

6. For each image, a separate plot is produced for every class in the training data, so all categories can be compared. These plots are most useful in two situations:
   When two classes have similar probabilities – they reveal which features may have caused the confusion.
   When the predicted class differs from the true class but the prediction or probability is not congruent with the visual evaluation, they show which features drove the switch compared with the original class.